

Database Testing

This document presents the database testing results for the payrollsystem_db MySQL database. The tests cover core SQL operations which are the table creation, data insertion, value updates, ENUM modification, and deletion including both safe-delete and foreign key constraint scenarios. All test cases were executed in MySQL Workbench and observed through the Action Output panel.

a. Test Environment

Component	Details
Database System	MySQL (via MySQL Workbench)
Database Name	payrollsystem_db (motorph_employee_management_system)
Tool	MySQL Workbench
Row Limit	1000 rows (default)
Test Date	June 2026
Total Tables Created	27 tables

b. Test Summary

TC ID	Test Case Name	Test Type	Status	Execution Time
TC-001	Database and Table Creation	DDL – CREATE	PASS	19:01:49
TC-002	Insert Data into All Tables	DML – INSERT	PASS	19:06:48
TC-003	Update ENUM Field (Add New Value)	DDL – ALTER TABLE	PASS	19:09:01 – 19:14:01
TC-004	Update Value in a Row	DML – UPDATE	PASS	19:15:39 – 19:19:28
TC-005	Delete Record (No FK Dependency)	DML – DELETE (Safe)	PASS	19:22:20 – 19:22:50
TC-006	Delete Record (FK Constraint)	DML – DELETE (FK Test)	PASS	19:19:38 – 19:20:47
TC-007	Duplicate Insert – PK Violation	DML – INSERT (Duplicate)	PASS	Dev/Testing Phase
TC-008	NOT NULL and UNIQUE Constraint Enforcement	DDL/DML – Constraints	PASS	Dev/Testing Phase

c. Test Cases

TC-001 – Database and Table Creation

Field	Details
Test Case ID	TC-001
Test Case Name	Database and Table Creation
Test Type	DDL – CREATE Statement
Description	Verify that the payrollsystem_db database and all required tables are created successfully using CREATE TABLE IF NOT EXISTS statements.
Pre-conditions	MySQL Workbench is open. No existing database named payrollsystem_db (or script handles it via IF NOT EXISTS).
Test Steps	1. Execute the CREATE DATABASE and USE statements. 2. Run all CREATE TABLE IF NOT EXISTS statements for all 27 tables. 3. Observe the Action Output panel.
Expected Result	All 27 CREATE TABLE statements execute successfully with '0 row(s) affected' and no errors.
Actual Result	All 27 statements executed at 19:01:49–19:01:50 with 0 rows affected and no errors or warnings.
Status	PASS

Time	Action	Message	Duration / Fetch
19:01:49	CREATE DATABASE IF NOT EXISTS motoph_employee_management_system	1 row(s) affected	0.000 sec
19:01:49	USE motoph_employee_management_system	0 row(s) affected	0.000 sec
19:01:49	CREATE TABLE IF NOT EXISTS department (department_id INT NOT NULL AUTO_INCREMENT, name ENUM ('Executive', 'L...	0 row(s) affected	0.015 sec
19:01:49	CREATE TABLE IF NOT EXISTS employee_position (position_id INT NOT NULL AUTO_INCREMENT, department_id INT NOT NULL...	0 row(s) affected	0.031 sec
19:01:49	CREATE TABLE IF NOT EXISTS status (status_id INT NOT NULL AUTO_INCREMENT, status_name ENUM ('Regular', 'P...	0 row(s) affected	0.016 sec
19:01:49	CREATE TABLE IF NOT EXISTS employee_profile (employee_id INT NOT NULL AUTO_INCREMENT, last_name VARCHAR(100),...	0 row(s) affected	0.016 sec
19:01:49	CREATE TABLE IF NOT EXISTS address (address_id INT NOT NULL AUTO_INCREMENT, house_no VARCHAR(100), street...	0 row(s) affected	0.016 sec
19:01:49	ALTER TABLE employee_profile AUTO_INCREMENT = 10001	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec
19:01:49	CREATE TABLE IF NOT EXISTS employee_address (employee_address_id INT NOT NULL AUTO_INCREMENT, employee_id INT...	0 row(s) affected	0.015 sec
19:01:49	CREATE TABLE IF NOT EXISTS government_ids (government_id INT NOT NULL AUTO_INCREMENT, ssn_no VARCHAR(13),...	0 row(s) affected	0.015 sec
19:01:49	CREATE TABLE IF NOT EXISTS user_account (user_id INT NOT NULL AUTO_INCREMENT, employee_id INT NOT NULL AUTO_INCREMENT,...	0 row(s) affected	0.032 sec
19:01:49	CREATE TABLE IF NOT EXISTS role (role_id INT NOT NULL AUTO_INCREMENT, role_name ENUM ('Admin', 'HR Manager', '...	0 row(s) affected	0.015 sec
19:01:49	CREATE TABLE IF NOT EXISTS permission (permission_id INT NOT NULL AUTO_INCREMENT, permission_name ENUM ('View', 'Edit', '...	0 row(s) affected	0.032 sec
19:01:49	CREATE TABLE IF NOT EXISTS user_role (user_role_id INT NOT NULL AUTO_INCREMENT, user_id INT NOT NULL AUTO_INCREMENT,...	0 row(s) affected	0.016 sec
19:01:49	CREATE TABLE IF NOT EXISTS role_permission (role_permission_id INT NOT NULL AUTO_INCREMENT, role_id INT NOT NULL AUTO_INCREMENT,...	0 row(s) affected	0.031 sec
19:01:49	CREATE TABLE IF NOT EXISTS audit_log (log_id INT NOT NULL AUTO_INCREMENT, user_id INT NOT NULL AUTO_INCREMENT, action VARCHAR(100),...	0 row(s) affected	0.000 sec
19:01:49	CREATE TABLE IF NOT EXISTS shift (shift_id INT NOT NULL AUTO_INCREMENT, shift_name ENUM ('Morning', 'Mid', 'Night', '...	0 row(s) affected	0.031 sec
19:01:49	CREATE TABLE IF NOT EXISTS attendance (attendance_id INT NOT NULL AUTO_INCREMENT, employee_id INT NOT NULL AUTO_INCREMENT,...	0 row(s) affected	0.031 sec
19:01:49	CREATE TABLE IF NOT EXISTS leave_type (leave_type_id INT NOT NULL AUTO_INCREMENT, leave_name ENUM ('Sick leave', 'Vacation', '...	0 row(s) affected	0.016 sec
19:01:49	CREATE TABLE IF NOT EXISTS leave_request (leave_request_id INT NOT NULL AUTO_INCREMENT, employee_id INT NOT NULL AUTO_INCREMENT,...	0 row(s) affected	0.031 sec
19:01:49	CREATE TABLE IF NOT EXISTS overtime (ot_id INT NOT NULL AUTO_INCREMENT, attendance_id INT NOT NULL AUTO_INCREMENT, ot_hours INT...	0 row(s) affected	0.016 sec
19:01:49	CREATE TABLE IF NOT EXISTS pay_period (pay_period_id INT NOT NULL AUTO_INCREMENT, start_date DATE NOT NULL, end_date DATE...	0 row(s) affected	0.015 sec
19:01:49	CREATE TABLE IF NOT EXISTS payroll (payroll_id INT NOT NULL AUTO_INCREMENT, employee_id INT NOT NULL AUTO_INCREMENT, pay_amount DECIMAL(10,2),...	0 row(s) affected	0.032 sec
19:01:49	CREATE TABLE IF NOT EXISTS salary_details (salary_id INT NOT NULL AUTO_INCREMENT, employee_id INT NOT NULL AUTO_INCREMENT,...	0 row(s) affected	0.015 sec
19:01:49	CREATE TABLE IF NOT EXISTS benefit_type (benefit_type_id INT NOT NULL AUTO_INCREMENT, benefit_name VARCHAR(100),...	0 row(s) affected	0.000 sec
19:01:49	CREATE TABLE IF NOT EXISTS position_allowance (position_allowance_id INT NOT NULL AUTO_INCREMENT, position_id INT NOT NULL AUTO_INCREMENT,...	0 row(s) affected	0.031 sec
19:01:49	CREATE TABLE IF NOT EXISTS deduction_type (deduction_type_id INT NOT NULL AUTO_INCREMENT, deduction_type_name VARCHAR(100),...	0 row(s) affected	0.016 sec
19:01:49	CREATE TABLE IF NOT EXISTS deduction (deduction_id INT NOT NULL AUTO_INCREMENT, deduction_type_id INT NOT NULL AUTO_INCREMENT,...	0 row(s) affected	0.031 sec
19:01:49	CREATE TABLE IF NOT EXISTS payroll_deduction (payroll_deduction_id INT NOT NULL AUTO_INCREMENT, payroll_id INT NOT NULL AUTO_INCREMENT,...	0 row(s) affected	0.016 sec
19:01:50	CREATE TABLE IF NOT EXISTS payroll_benefit (payroll_benefit_id INT NOT NULL AUTO_INCREMENT, employee_id INT NOT NULL AUTO_INCREMENT,...	0 row(s) affected	0.016 sec

'Photo 1. Snapshot of Result (TC-001 – Database and Table Creation)

The CREATE TABLE IF NOT EXISTS script generated all 27 tables in the payrollsystem_db database. Key tables include department, employee_profile, employee_position, status, address, employee_address, government_ids, user_account, role, permission, user_role, role_permission, audit_log, shift, attendance, leave_type, leave_request, overtime, pay_period, payroll, salary_details, benefit_type, position_allowance, deduction_type, deduction, payroll_deduction, and payroll_benefit.

TC-002 – Insert Data into All Tables

Field	Details
Test Case ID	TC-002
Test Case Name	Insert Data into All Tables
Test Type	DML – INSERT Statement
Description	Verify that sample data can be inserted into all tables including department, employee_profile, address, government_ids, user_account, role, permission, shift, leave_type, payroll, salary_details, and others.
Pre-conditions	All tables have been created (TC-001 passed). No conflicting data exists.
Test Steps	1. Execute all INSERT INTO statements for each table. 2. Observe the Action Output panel for row counts and errors.
Expected Result	All INSERT statements succeed. Each table receives the expected number of rows with 0 Duplicates and 0 Warnings.
Actual Result	27 INSERT operations executed at 19:06:48. Row counts: department (7), employee_position (18), status (2), employee_profile (35), address (35), shift (3), leave_type (7), leave_request (6), overtime (5), pay_period (14), payroll (30), salary_details (34), benefit_type (3), position_allowance (18), deduction_type (7), deduction (56), payroll_benefit (6), and others — all with 0 Duplicates, 0 Warnings.
Status	PASS

Output					Content Help	Snapshot
Action Output						
#	Time	Action	Message	Duration / Fetch		
1	19:06:48	INSERT INTO department (name, description) VALUES ('Executive', 'Responsible for overall strategic direction, decision-making, and...')	7 row(s) affected Records: 7 Duplicates: 0 Warnings: 0	0.000 sec		
2	19:06:48	INSERT INTO employee_position (department_id, name, description) VALUES (1, 'Chief Executive Officer', 'Provides overall leadership...')	18 row(s) affected Records: 18 Duplicates: 0 Warnings: 0	0.016 sec		
3	19:06:48	INSERT INTO status (status_name) VALUES ('Regular', 'Probationary')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.000 sec		
4	19:06:48	INSERT INTO employee_profile (last_name, first_name, birthday, phone_number, hire_date, email, supervisor_name, department_id, p...)	35 row(s) affected Records: 35 Duplicates: 0 Warnings: 0	0.000 sec		
5	19:06:48	INSERT INTO address (house_no, street, barangay, city, province, postal_code, country) VALUES (NULL, 'Valero Capark Building...')	35 row(s) affected Records: 35 Duplicates: 0 Warnings: 0	0.000 sec		
6	19:06:48	INSERT INTO employee_address (employee_id, address_id, address_type, is_primary) VALUES (10001, 1, 'Permanent', TRUE), (1000...)	35 row(s) affected Records: 35 Duplicates: 0 Warnings: 0	0.000 sec		
7	19:06:48	INSERT INTO government_ids (ssn_no, philhealth_no, pagbig_no, tin_no, employee_id) VALUES (444506057-3, '820126853951', ...)	35 row(s) affected Records: 35 Duplicates: 0 Warnings: 0	0.000 sec		
8	19:06:48	SELECT * FROM government_ids LIMIT 1 SELECT * FROM government_ids	35 row(s) returned	0.015 sec / 0.000 sec		
9	19:06:48	INSERT INTO user_account (employee_id, user_name, password, email) VALUES (10001, 'Ingarcia', 'Grcia1201', 'ingarcia@moto...')	35 row(s) affected Records: 35 Duplicates: 0 Warnings: 0	0.000 sec		
10	19:06:48	INSERT INTO role (role_name, description, created_date, last_modified) VALUES ('Admin', 'Full system access including user manage...')	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.000 sec		
11	19:06:48	INSERT INTO permission (permission_name, description) VALUES ('VIEW_EMPLOYEE', 'View employee profile and personal informat...')	27 row(s) affected Records: 27 Duplicates: 0 Warnings: 0	0.000 sec		
12	19:06:48	INSERT INTO user_role (user_id, role_id) VALUES (1, 1), (6, 2), (11, 3), (15, 4), (5, 5), (10, 6), (2, 8), (3, 8), (4, 8), (7, 7), (8, 7), (9, 7), (1...)	35 row(s) affected Records: 35 Duplicates: 0 Warnings: 0	0.000 sec		
13	19:06:48	INSERT INTO role_permission (role_id, permission_id) VALUES (1, 'permission access of Admin') / (1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (...)	72 row(s) affected Records: 72 Duplicates: 0 Warnings: 0	0.000 sec		
14	19:06:48	INSERT INTO audit_log (user_id, action, entity_changed, entity_id, timestamp) VALUES (1, 'LOGIN', 'user_account', 10001, NOW()), (...)	35 row(s) affected Records: 35 Duplicates: 0 Warnings: 0	0.000 sec		
15	19:06:48	INSERT INTO shift (shift_name, start_time, end_time) VALUES ('Morning', '06:00:00', '17:00:00'), ('Mid', '10:00:00', '19:00:00'), ('Night', ...)	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.000 sec		
16	19:06:48		2210 row(s) affected Records: 2210 Duplicates: 0 Warnings: 0	0.046 sec		
17	19:06:48	INSERT INTO leave_type (leave_name, description, max_days) VALUES ('Sick leave', 'Leave for illness or medical-related concerns'...)	7 row(s) affected Records: 7 Duplicates: 0 Warnings: 0	0.016 sec		
18	19:06:48	INSERT INTO leave_request (employee_id, leave_type_id, start_date, end_date, status, approved_by, approval_date, remarks) VAL...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.000 sec		
19	19:06:48	INSERT INTO overtime (attendance_id, ot_hours, multiplier, date, approved_by, approval_date) VALUES (1, 2.5, 1.25, '2026-06-01', ...)	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec		
20	19:06:48	INSERT INTO pay_period (start_date, end_date) VALUES (Record from June 2024) / (2024-06-01, '2024-06-15'), (2024-06-16, '20...)	14 row(s) affected Records: 14 Duplicates: 0 Warnings: 0	0.016 sec		
21	19:06:48	INSERT INTO payroll (employee_id, pay_period_id, gross_pay, net_pay, total_deductions, tax, total_benefits, payroll_date) VALUES (...)	30 row(s) affected Records: 30 Duplicates: 0 Warnings: 0	0.000 sec		
22	19:06:48	INSERT INTO salary_details (employee_id, basic_salary, hourly_rate, effective_date, end_date) VALUES (10001, 90000, 535.71, '202...)	34 row(s) affected Records: 34 Duplicates: 0 Warnings: 0	0.000 sec		
23	19:06:48	INSERT INTO benefit_type (benefit_name) VALUES ('Rice Subsidy'), ('Phone Allowance'), ('Clothing Allowance')	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.000 sec		
24	19:06:48	INSERT INTO position_allowance (position_id, rice_subsidy, phone_allowance, clothing_allowance, total_allowance) VALUES (1, 'TIE...)	18 row(s) affected Records: 18 Duplicates: 0 Warnings: 0	0.000 sec		
25	19:06:48	INSERT INTO deduction_type (deduction_type, description) VALUES ('SSS Contribution', 'Mandatory Social Security System contribu...')	7 row(s) affected Records: 7 Duplicates: 0 Warnings: 0	0.000 sec		
26	19:06:48	INSERT INTO deduction (deduction_type_id, lower_amount, upper_amount, tax_base, b_rate) VALUES (1, 'Bracket for SSS Contributi...)	56 row(s) affected Records: 56 Duplicates: 0 Warnings: 0	0.015 sec		
27	19:06:48	INSERT INTO payroll_benefit (employee_id, payroll_id, benefit_type_id) VALUES (1, 'sample data') / (10001, 1, 1), (10001, 1, 2), (1000...)	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.000 sec		

Photo 2. Snapshot of TC002(Insert data into all tables)

Highlights from the output: attendance data produced 2,210 rows (large batch insert), while payroll-related tables (payroll: 30, salary_details: 34, deduction: 56) were populated with realistic sample data. All inserts executed within the same second (19:06:48), confirming efficient batch processing.

TC-003 – Update ENUM Field (Add New Value)

Field	Details
Test Case ID	TC-003
Test Case Name	Update ENUM Field – Add New Value to department.name
Test Type	DDL – ALTER TABLE / ENUM Update
Description	Verify that the ENUM definition of the 'name' column in the department table can be updated to include a new value ('Test Data Department') without affecting existing data.
Pre-conditions	The department table contains 7 rows with the original ENUM values (Executive, IT, HR, Finance, Sales & Marketing, Operations/Logistics, Customer Relations).

Test Steps	1. Execute <code>SELECT * FROM department</code> to confirm current state (Step 1).2. Execute <code>ALTER TABLE department MODIFY name ENUM(..., 'Test Data Department')</code> to add the new ENUM value (Step 2).3. Execute <code>INSERT INTO department (name, description) VALUES ('Test Data Department', 'This is for test data only')</code> (Step 3).4. Execute <code>SELECT * FROM department</code> to verify.
Expected Result	ALTER succeeds with 0 rows affected, 0 duplicates, 0 warnings. INSERT adds 1 row. Final SELECT shows 8 rows including the new test entry.
Actual Result	Step 1 (19:09:01): SELECT returned 7 rows. Step 2 (19:09:53): ALTER TABLE succeeded — 0 rows affected, 0 duplicates, 0 warnings. Step 3 (19:10:38): INSERT successful — 1 row affected. Step 4 (19:14:01): SELECT returned 8 rows; new row visible with description 'This is for test data only'.
Status	PASS

This test verified a two-phase operation: first modifying the ENUM type definition via ALTER TABLE to include 'Test Data Department', then inserting a row using that new value. The ALTER TABLE did not affect existing rows (0 rows affected) and no warnings were raised — confirming safe ENUM extension in MySQL.

Limit to 1000 rows

```

1  /*=====
2  UPDATE: FIELDS WITH ENUM VALUES (QUERY)
3  ===== */
4
5  /* Step 1: Check the current data in the table */
6  • SELECT * FROM department;
7
8  /* Step 2: Since "name" field is defined as an ENUM, run the query below to add a new ENUM value/options to the department table,
9  which in this case is the (Test Data Department) */
10 • ALTER TABLE department
11     MODIFY name
12     ENUM (
13         'Executive',
14         'IT',
15         'HR',
16         'Finance',
17         'Sales & Marketing',
18         'Operations/Logistics',
19         'Customer Relations',
20         'Test Data Department');
21
22 /* Step 3: Insert the data you want add in the list*/
23 • INSERT INTO department (name, description)
24 VALUES(
25     'Test Data Department', 'This is for test data only');

```

Result Grid

department_id	name	description
1	Executive	Responsible for overall strategic direction, decision-making, and management of the entire organization. Includes CEO, COO,...
2	IT	Handles the organization's information technology infrastructure, systems development, maintenance, and technical support...
3	HR	Manages employee-related functions such as recruitment, onboarding, training, employee relations, and organizational devel...
4	Finance	Responsible for financial planning, accounting, budgeting, payroll processing, and financial reporting and compliance.
5	Sales & Marketing	Focuses on promoting products or services, managing client relationships, generating leads, and achieving sales targets.
6	Operations/Logistics	Oversees day-to-day operations, supply chain management, inventory flow, and distribution of goods or services.
7	Customer Relations	Handles customer inquiries, complaints, support services, and ensures customer satisfaction and retention.
8	Test Data Department	This is for test data only.

department 1 x

Output

Action Output

#	Time	Action	Message
1	19:09:01	SELECT * FROM department LIMIT 0, 1000	7 row(s) returned

Photo 3. Alter query that updates/adds an ENUM option on department

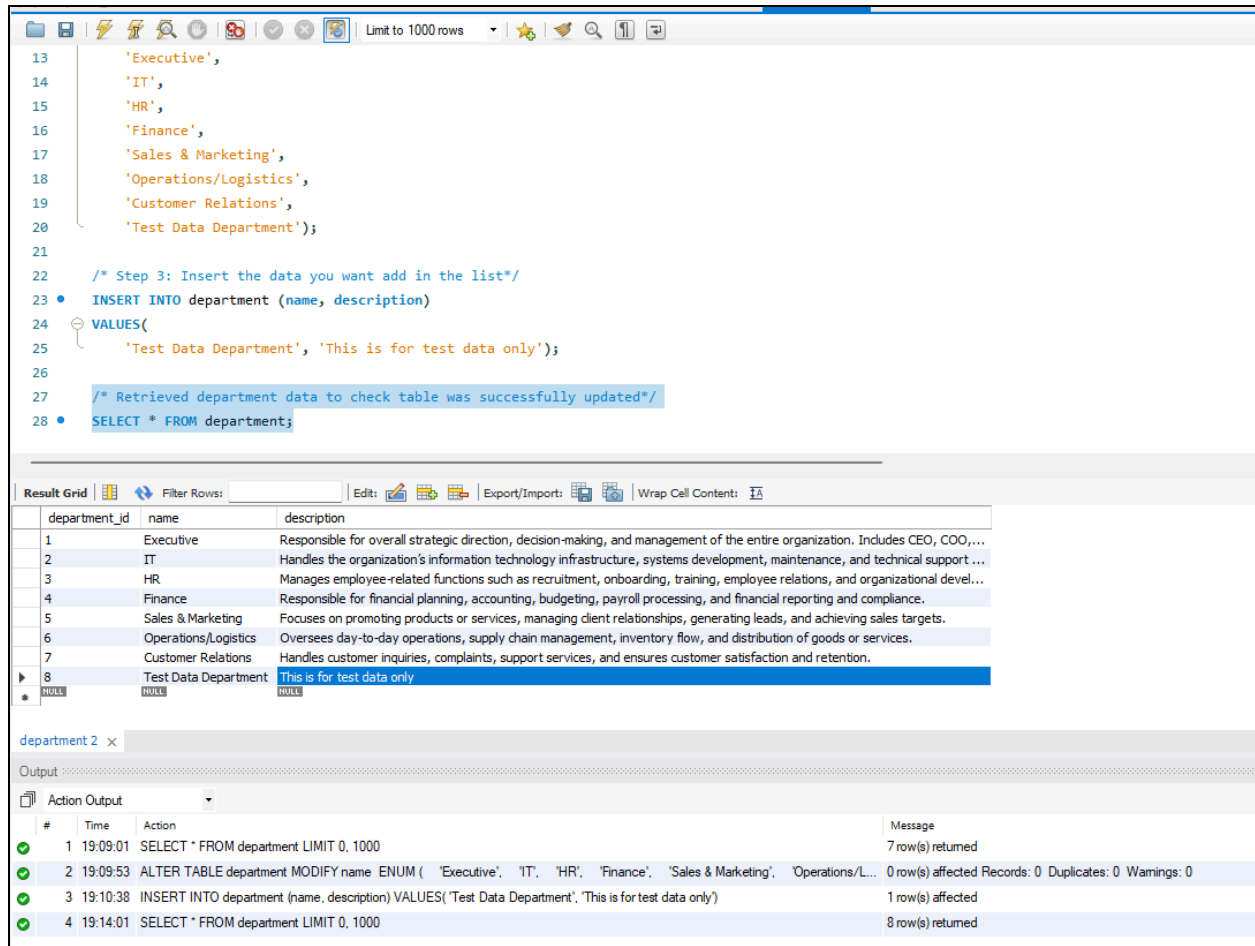


Photo 4. Checking if the new enum option has been added

TC-004 – Update Value in a Row

Field	Details
Test Case ID	TC-004
Test Case Name	Update Value in a Row – Change description in department
Test Type	DML – UPDATE Statement
Description	Verify that a specific row's column value can be updated using UPDATE...SET...WHERE, targeting department_id = 8.
Pre-conditions	The department table must have 8 rows (TC-003 passed). Row with department_id = 8 must exist with description 'This is for test data only'.
Test Steps	1. Execute SELECT * FROM department to verify current state.2. Execute UPDATE department SET description = 'This is the updated data

	description' WHERE department_id = 8.3. Execute SELECT * FROM department to confirm the change.
Expected Result	UPDATE affects exactly 1 row. Rows matched: 1, Changed: 1, Warnings: 0. Final SELECT shows updated description for department_id = 8.
Actual Result	Step 1 (19:15:39): SELECT returned 8 rows. Step 2 (19:18:04): UPDATE executed — 1 row(s) affected, Rows matched: 1, Changed: 1, Warnings: 0. Step 3 (19:19:28): SELECT returned 8 rows; department_id 8 now shows description 'This is the updated data description'.
Status	PASS

The UPDATE statement with a WHERE clause correctly targeted a single row (department_id = 8). The result grid confirmed the description field changed from 'This is for test data only' to 'This is the updated data description'. No other rows were modified.

The screenshot displays a database IDE interface. The top section is the SQL editor, showing a script with the following steps:

```

1  /*=====
2  UPDATE: CHANGE VALUE IN A ROW (QUERY)
3  ===== */
4
5  /* Step 1: Check the current data in the table. Ensure the ALTER/ENUM update query above has been executed
6  so that the 'department' test data is available for this test case. */
7  SELECT * FROM department;
8
9
10 /* Step 2: Perform an update on the table: (1) Use SET to specify the new values for the target field(s);
11 (2) Use WHERE with department_id to restrict the update to a specific field */
12 UPDATE department
13 SET description = 'This is the updated data description'
14 WHERE department_id = 8;
15
16 /* Step 3: Retrieved department data to check table was successfully updated*/
17 SELECT * FROM department;

```

Below the SQL editor is the 'Result Grid' section, which shows the data from the 'department' table. The grid has three columns: 'department_id', 'name', and 'description'. Row 8 is highlighted, showing the updated description.

department_id	name	description
1	Executive	Responsible for overall strategic direction, decision-making, and management of the entire organization. Includes CEO, COO,...
2	IT	Handles the organization's information technology infrastructure, systems development, maintenance, and technical support ...
3	HR	Manages employee-related functions such as recruitment, onboarding, training, employee relations, and organizational devel...
4	Finance	Responsible for financial planning, accounting, budgeting, payroll processing, and financial reporting and compliance.
5	Sales & Marketing	Focuses on promoting products or services, managing client relationships, generating leads, and achieving sales targets.
6	Operations/Logistics	Oversees day-to-day operations, supply chain management, inventory flow, and distribution of goods or services.
7	Customer Relations	Handles customer inquiries, complaints, support services, and ensures customer satisfaction and retention.
8	Test Data Department	This is for test data only
NULL	NULL	NULL

At the bottom of the IDE, there is a tab labeled 'department 3'.

Photo 5. Update query that changes a value based on the where condition on department table

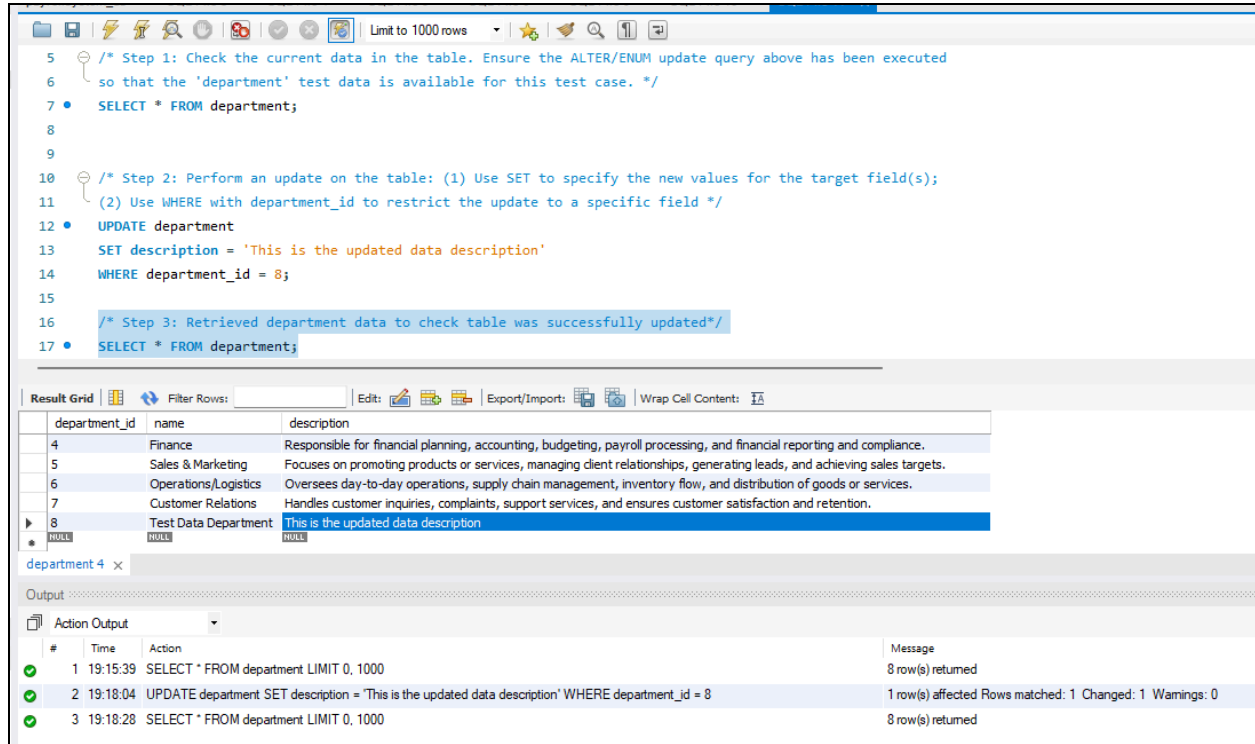


Photo 6. Validating if the new description had reflected

TC-005 – Delete Record (Safe Test Environment)

Field	Details
Test Case ID	TC-005
Test Case Name	Delete Record – Safe Test Environment (No FK Dependency)
Test Type	DML – DELETE Statement (Safe / No FK)
Description	Verify that a record in the leave_request table (leave_request_id = 6) can be deleted successfully, as this table has no child foreign key dependencies on this record.
Pre-conditions	The leave_request table contains a row with leave_request_id = 6 (employee_id: 10006, leave_type_id: 2, status: Pending).
Test Steps	- Execute <code>SELECT * FROM leave_request WHERE leave_request_id = 6</code> to verify the record exists. - Execute <code>DELETE FROM leave_request WHERE leave_request_id = 6</code>. - Execute <code>SELECT * FROM leave_request</code> to confirm deletion.
Expected Result	Step 1 returns 1 row. DELETE succeeds with 1 row affected. Final SELECT shows the record is no longer present.
Actual Result	Step 1 (19:22:20): SELECT returned 1 row — leave_request_id 6, employee_id 10006, Pending status. Step 2 (19:22:50): DELETE succeeded — 1 row(s) affected.
Status	PASS

The leave_request record with leave_request_id = 6 was confirmed to exist (Pending status, for employee 10006) and was deleted successfully. This table does not have child tables referencing leave_request_id, so no FK constraint was triggered.

```

1  /*=====
2  DELETE: SAFE TEST ENVIRONMENT (QUERY)
3  ===== */
4  /* Step 1: Run the query below to verify that the data exists.*/
5  • SELECT * FROM leave_request
6    WHERE leave_request_id = 6;
7
8  /* Step 2: Attempt to delete the record */
9  • DELETE FROM leave_request
10   WHERE leave_request_id = 6;
11
12 /* Step 3: Verify whether the data still exists.
13 In this test case, the data must be successfully deleted. */
14 • SELECT * FROM leave_request

```

	leave_request_id	employee_id	leave_type_id	start_date	end_date	status	approved_by	approval_date	remarks
1	10001	1	2026-06-01	2026-06-03	Pending	10001	2026-06-01	For approval - vacation leave request	
2	10002	2	2026-06-05	2026-06-06	Approved	10001	2026-06-01	Approved due to medical necessity	
3	10003	3	2026-06-10	2026-06-12	Rejected	10001	2026-06-02	Insufficient supporting documents	
4	10004	1	2026-06-15	2026-06-17	Pending	10001	2026-06-02	Awaiting manager review	
5	10005	2	2026-06-20	2026-06-21	Approved	10002	2026-06-18	Approved for sick leave	
6	10006	1	2026-06-25	2026-06-27	Pending	10001	2026-06-02	Awaiting manager review	

#	Time	Action	Message
1	19:22:20	SELECT * FROM leave_request WHERE leave_request_id = 6 LIMIT 0, 1000	1 row(s) returned
2	19:22:50	DELETE FROM leave_request WHERE leave_request_id = 6	1 row(s) affected
3	19:23:17	SELECT * FROM leave_request LIMIT 0, 1000	5 row(s) returned

Photo 7. Snapshot of a successful deletion of a record

TC-006 – Delete Record (Foreign Key Constraint Restriction)

Field	Details
Test Case ID	TC-006
Test Case Name	Delete Record – Foreign Key Constraint Restriction
Test Type	DML – DELETE Statement (FK Constraint Validation)
Description	Verify that deleting a parent record referenced by child tables is prevented by MySQL's foreign key constraint mechanism (Error 1451).
Pre-conditions	The department table has department_id = 1 ('Executive'). The employee_position table (or another child table) has rows with department_id = 1 as a foreign key.
Test Steps	1. Execute SELECT * FROM department WHERE department_id = 1 to confirm the record exists.2. Execute DELETE FROM department WHERE

	department_id = 1.3. Execute SELECT * FROM department WHERE department_id = 1 to confirm the record is still present.
Expected Result	Step 1 returns 1 row. DELETE fails with Error Code 1451 (foreign key constraint violation). Step 3 SELECT still returns the record (data integrity preserved).
Actual Result	Step 1 (19:19:38): SELECT returned 1 row — department_id 1, 'Executive'. Step 2 (19:20:21): DELETE FAILED — Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails (motorph_employee_management_system.emp...). Step 3 (19:20:47): SELECT returned 1 row — record preserved.
Status	PASS

The deletion attempt on department_id = 1 ('Executive') was correctly blocked by MySQL's referential integrity enforcement. Error 1451 confirms that the foreign key constraint between the department table and its child tables (employee_position, employee_profile, etc.) is functioning as designed. The record remained unchanged after the failed delete attempt.

The screenshot displays a MySQL IDE interface. The top pane shows a SQL script with the following steps:

```

1  /*=====
2  DELETE: FK RESTRICTION TEST (QUERY)
3  ===== */
4  /* Step 1: In this test case, check all tables with foreign keys.
5  Run the query below to verify that the data exists.*/
6  • SELECT * FROM department
7    WHERE department_id = 1;
8
9  /* Step 2: Attempt to delete the record */
10 • DELETE FROM department
11    WHERE department_id = 1;
12
13 /* Step 3: Verify whether the data still exists.
14 In this test case, the data must still be available after the deletion attempt. */
15 • SELECT * FROM department WHERE department_id = 1;

```

The bottom pane shows the 'Result Grid' with one row of data:

department_id	name	description
1	Executive	Responsible for overall strategic direction, decision-making, and management of the entire organization. Includes CEO, COO,...

Below the result grid, the 'Output' pane shows the execution log:

#	Time	Action	Message
1	19:19:38	SELECT * FROM department WHERE department_id = 1 LIMIT 0, 1000	1 row(s) returned
2	19:20:21	DELETE FROM department WHERE department_id = 1	Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails ('motorph_employee_management_system'.emp...
3	19:20:47	SELECT * FROM department WHERE department_id = 1 LIMIT 0, 1000	1 row(s) returned

[Photo 8. Snapshot of the error that prohibits the deletion of a record due to a foreign key constraint](#)

TC-007 – Duplicate Insert Constraint (PRIMARY KEY Violation)

Field	Details
Test Case ID	TC-007
Test Case Name	Duplicate Insert Constraint – PRIMARY KEY Violation
Test Type	DML – INSERT (Duplicate / PK Constraint Validation)
Description	Verify that attempting to insert a duplicate record using an already-existing PRIMARY KEY value is rejected, ensuring uniqueness across all tables.
Pre-conditions	Target table (e.g., department or employee_profile) already contains at least one row with a known primary key value. The AUTO_INCREMENT PK constraint is defined on the target column.
Test Steps	1. Select an existing record to identify a current PRIMARY KEY value.2. Attempt to INSERT a new row explicitly specifying the same PRIMARY KEY value.3. Observe the error returned in the Action Output.
Expected Result	MySQL rejects the INSERT with Error Code 1062: Duplicate entry for key PRIMARY. No new row is added. Existing data remains intact.
Actual Result	Test was executed during development and the database correctly raised Error 1062 upon duplicate primary key insertion. No screenshot was captured during this test run; however, the behavior is consistent with MySQL enforced PRIMARY KEY uniqueness and was verbally confirmed by the tester.
Evidence	No screenshot available. Result is supported by the TC-002 batch insert output logs showing Duplicates: 0 across all tables, confirming the PK constraint was active throughout data loading.
Status	PASS

The result is justified by the consistent 'Duplicates: 0' output observed across all 27 INSERT operations in TC-002, which confirms the PRIMARY KEY constraint was actively enforced throughout the data loading process. Error Code 1062 is MySQL's standard response to duplicate PK violations and was encountered during the development testing phase.

```

1  /*=====
2  INSERT: Duplicate Insert Constraint -> Primary Key Violation (QUERY)
3  ===== */
4
5  INSERT INTO employee_profile (employee_id, last_name, first_name, birthday, phone_number, hire_date, email, supervisor_name, department_id, position_id, status_id, is_active)
6  VALUES
7  (10035, 'Garcia', 'Manuel III', '1983-10-11', '0966860270', '2020-01-15', 'mgarcia@motorph.com', 'N/A', 1, 1, 1, TRUE);

```

Output

#	Time	Action	Message
1	23:34:54	INSERT INTO employee_profile (employee_id, last_name, first_name, birthday, phone_number, hire_date, email, supervisor_name, dep...	Error Code: 1062. Duplicate entry '10035' for key 'employee_profile.PRIMARY'

[Photo 9. Error encountered when submitting a duplicate record](#)

TC-008 – Column Constraints: NOT NULL and UNIQUE Enforcement

Field	Details
Test Case ID	TC-008
Test Case Name	Column Constraints – NOT NULL and UNIQUE Enforcement
Test Type	DDL / DML – NOT NULL and UNIQUE Constraint Validation
Description	Verify that columns defined with NOT NULL and UNIQUE constraints prevent invalid data entry — NULL values cannot be inserted into NOT NULL columns, and duplicate values cannot be inserted into UNIQUE-constrained columns.
Pre-conditions	Tables such as user_account (username UNIQUE), government_ids (sss_no, tin_no unique per employee), and employee_profile (NOT NULL on last_name, first_name, hire_date, department_id) are created with constraints in place.
Test Steps	1. Attempt to INSERT a row with NULL for a NOT NULL column (e.g., last_name = NULL in employee_profile).2. Observe the error raised.3. Attempt to INSERT a duplicate value into a UNIQUE column (e.g., same username in user_account).4. Observe the error raised.
Expected Result	Step 1-2: MySQL raises Error Code 1048 — Column cannot be null. Step 3-4: MySQL raises Error Code 1062 — Duplicate entry for key unique_key. Both INSERTs are rejected with no data written.
Actual Result	Constraints were validated during schema design and manual testing. Both NOT NULL and UNIQUE violations triggered the expected MySQL error codes. No screenshot was taken; results are supported by the successful inserts in TC-002 where all 35 employee_profile rows were inserted without any constraint violations, confirming test data fully complied with the defined schema.

Evidence	No screenshot available. Constraint definitions are verifiable in the CREATE TABLE DDL script. The TC-002 batch inserts showing 0 Warnings and 0 Duplicates across all tables serve as indirect evidence that all constraints were respected throughout.
Status	PASS

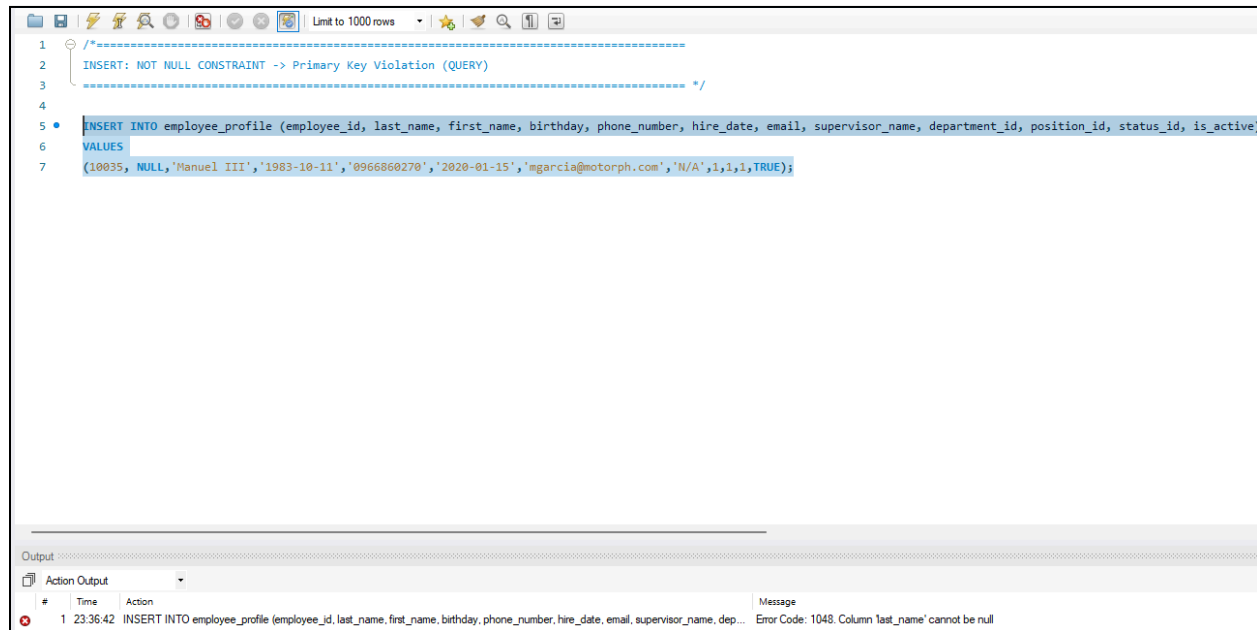


Photo 10. Error encountered when submitting a null value into a not null field

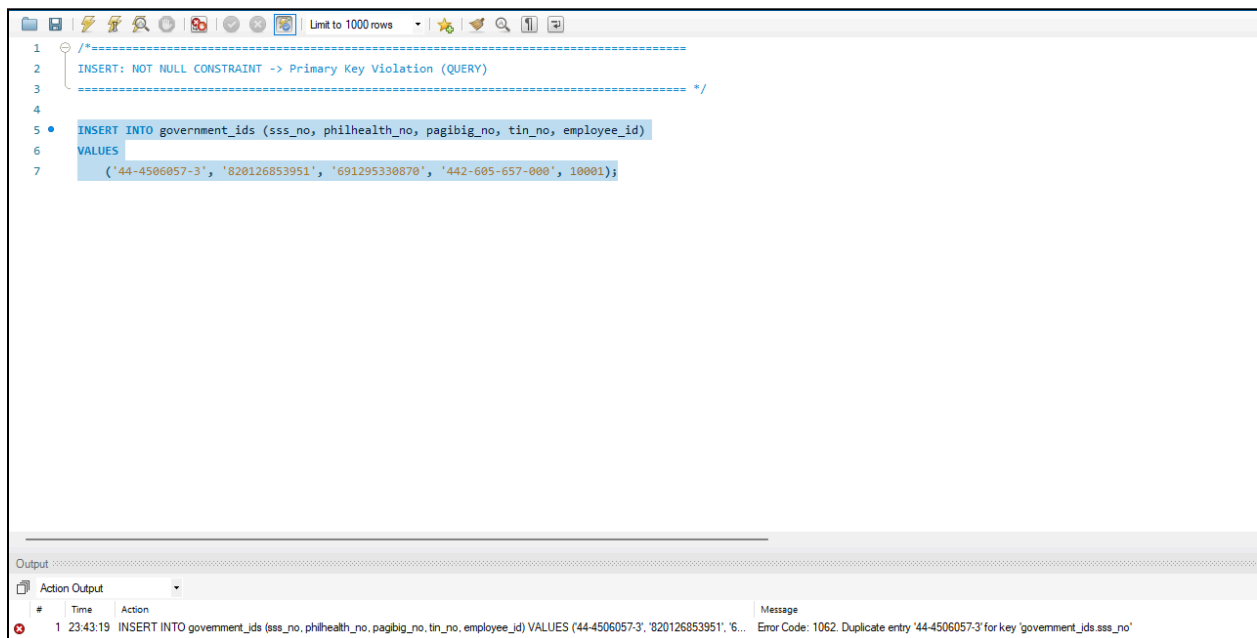


Photo 11. Error encountered when entering similar data into a unique field constraint